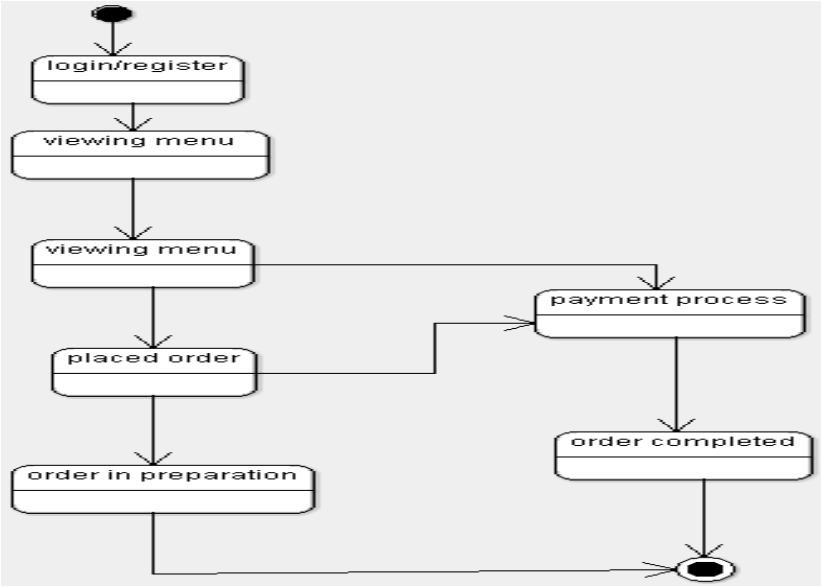
NAME: SHAMINI C
SPR NO: 9490

EXP NO: 6 - DRAW RELEVANT STATE CHART AND ACTIVITY DIAGRAMS FOR THE SAME SYSTEM

PROJECT TITLE: Smart Canteen Management System

DESCRIPTION: STATE CHART DIAGRAM

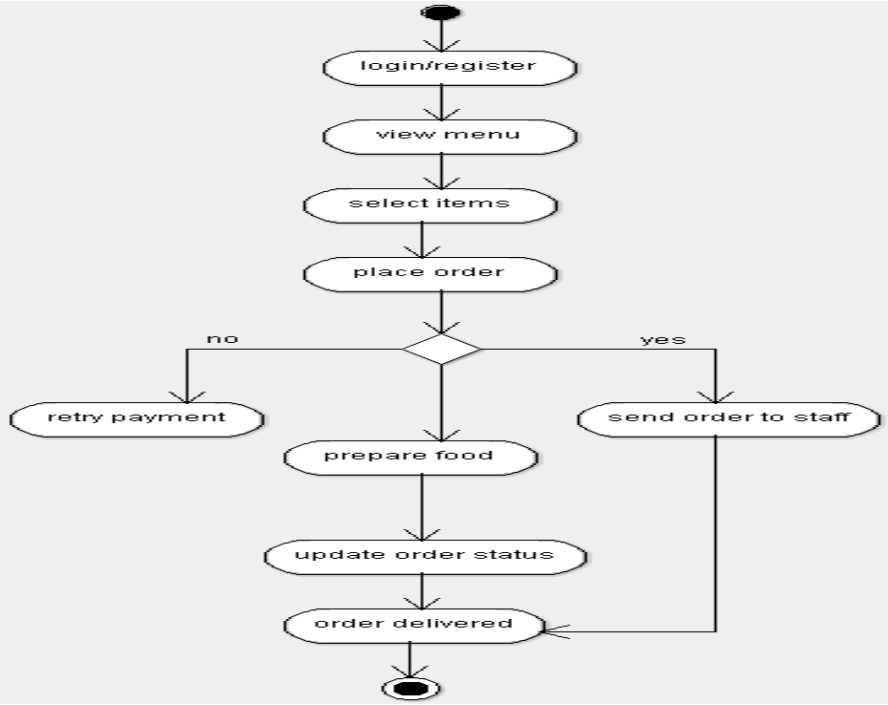
- The system starts at the Initial State.
- It moves to the Idle State, where it waits for user interaction.
- When the user selects an option, the system transitions to the Menu Display State to show available food items.
- If the user chooses to order, the system enters the Order Selection State, where the user selects the item and enters quantity.
- After input, the system moves to the Order Processing State, where it retrieves price and calculates the total amount.
- If the item exists, the system transitions to the Order Confirmation State, where:
- Order is stored in the database
- Confirmation message is displayed



NAME: SHAMINI C
SPR NO: 9490

DESCRIPTION : ACTIVITY DIAGRAM

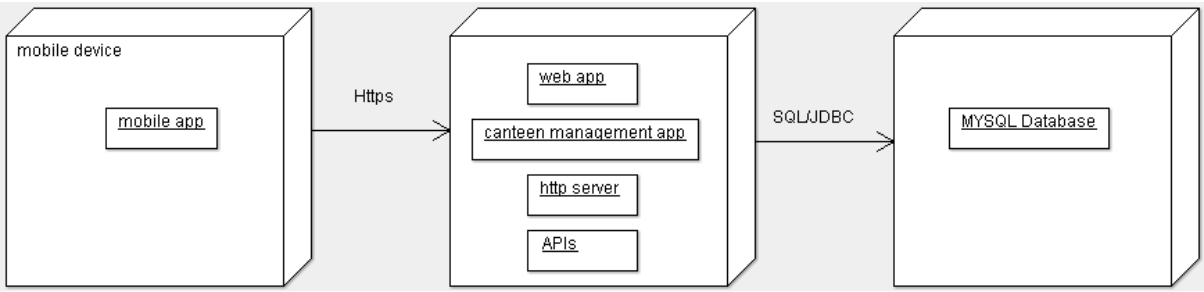
- An Activity Diagram represents the flow of activities in a system from start to end.
- It shows the sequence of actions, decisions, and parallel processes involved in the system.
- In the Smart Canteen Management System, the process starts with the customer logging into the system.
- The customer then views the menu and selects food items to place an order.
- The system processes the order and sends it to the staff for preparation.
- The staff prepares the food and updates the order status.
- The system then processes the payment made by the customer.
- After successful payment, the order is confirmed and delivered to the customer.
- Finally, the process ends after order completion.



NAME: SHAMINI C
SPR NO: 9490

DESCRIPTION : DEPLOYMENT DIAGRAM

- The deployment diagram represents the physical architecture of the Smart Canteen Management System.
- The system consists of a Client Node (User System) where the application is accessed by users.
- The client node runs the Java application interface for menu viewing and order placement.
- The client connects to the Application Layer using JDBC for database communication.
- A Database Server Node is used to store all system data such as menu and orders.
- The database server runs SQL Server, which manages data storage and retrieval.
- The client and database server communicate through a network connection.
- The system ensures smooth data flow between the user interface and the database.
- All order processing and data storage operations are handled by the database server.
- The deployment diagram shows how hardware and software components are distributed in the system.



RESULT:

Thus, the Activity Diagram and State Chart Diagram for the Smart canteen Management System were successfully developed to represent the workflow and state transitions of the system.

NAME: SHAMINI C
SPR NO: 9490

EXPNO: 07 IMPLEMENT THE SYSTEM AS PER THE DETAILED DESIGN

PROJECT TITLE: Smart Canteen Management System

1. TECHNOLOGY STACK

- User Interface: Java Swing (GUI)
- Programming Language: Java
- Database: SQL Server
- JDBC: SQL Server Driver
- Software: VS Code, SQL Server Management Studio (SSMS)
- Hardware: Minimum 4GB RAM

2. IMPLEMENTATION STEPS

1. Open SQL Server Management Studio (SSMS)

SQL Server Database

```
SELECT *

FROM (VALUES

('Shamini','Dosa',50,'Ordered'),

('Kavi Sri','Idli',30,'Ordered'),

('Keerthi','Pongal',40,'Pending'),

('Janani','Meals',80,'Ordered'),

('Saranya','Fried Rice',90,'Ordered')

) AS canteen(name,item,price,status)

 order status verification query

SELECT name, item,

CASE
```

```
WHEN status='Ordered' THEN 'Confirmed'

ELSE 'Waiting'

END AS order_status

FROM (VALUES

('Shamini','Dosa',50,'Ordered'),

('Kavi Sri','Idli',30,'Ordered'),

('Keerthi','Pongal',40,'Pending'),

('Janani','Meals',80,'Ordered'),

('Saranya','Fried Rice',90,'Ordered')

) AS canteen(name,item,price,status);
```

Total bill calculation query:

```
SELECT SUM(price) AS total_amount

FROM (VALUES

('Shamini','Dosa',50,'Ordered'),

('Kavi Sri','Idli',30,'Ordered'),

('Keerthi','Pongal',40,'Pending'),

('Janani','Meals',80,'Ordered'),

('Saranya','Fried Rice',90,'Ordered')

) AS canteen(name,item,price,status);
```

Database Connection Using JDBC

```
import java.sql.*;

public class DBConnection {

    public static Connection getConnection() {

        try {
```

```
            Class.forName("com.microsoft.sqlserver.jdbc.SQLServerDriver");
            Connection con = DriverManager.getConnection(

"jdbc:sqlserver://localhost;databaseName=tempdb;encrypt=true;trustServerCertificate=true;integratedSecurity=true"
            );
            System.out.println("Connected");
            return con;
        } catch (Exception e) {
            System.out.println("ERROR: " + e);
            return null;
        }
    }
}
```

Core Classes

1. Order Class

```
public class Order {
    public void placeOrder(String name, String item, int price) {
        System.out.println("Order placed by: " + name);
        System.out.println("Item: " + item);
        System.out.println("Price: " + price);
    }
}
```

2. Verification Class

```
public class Verification {
    public boolean checkUser(String name) {
        return name.equals("Shamini") ||
               name.equals("Kavi Sri") ||
               name.equals("Keerthi") ||
               name.equals("Janani") ||
               name.equals("Saranya");
    }
}
```

GUI IMPLEMENTATION

```
import java.awt.*;
import javax.swing.*;
public class CanteenGUI {
    public static void main(String[] args) {
        JFrame f = new JFrame("Smart Canteen System");
        f.setSize(350, 250);
        f.setLayout(new GridLayout(4,2));
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        JTextField name = new JTextField();
        JTextField item = new JTextField();
        JButton submit = new JButton("Place Order");
        f.add(new JLabel("Name"));
        f.add(name);
```

```
        f.add(new JLabel("Item"));
        f.add(item);
        f.add(new JLabel(""));
        f.add(submit);
        submit.addActionListener(e -> {
            Verification v = new Verification();
            boolean valid = v.checkUser(name.getText());
            if(valid) {
                new Order().placeOrder(name.getText(), item.getText(), 50);
                JOptionPane.showMessageDialog(f,
                    "Order Placed Successfully\nItem: " + item.getText() + "\nPrice: 50");
            } else {
                JOptionPane.showMessageDialog(f, "Invalid User");
            }
        });
        f.setVisible(true);
    }
}
```

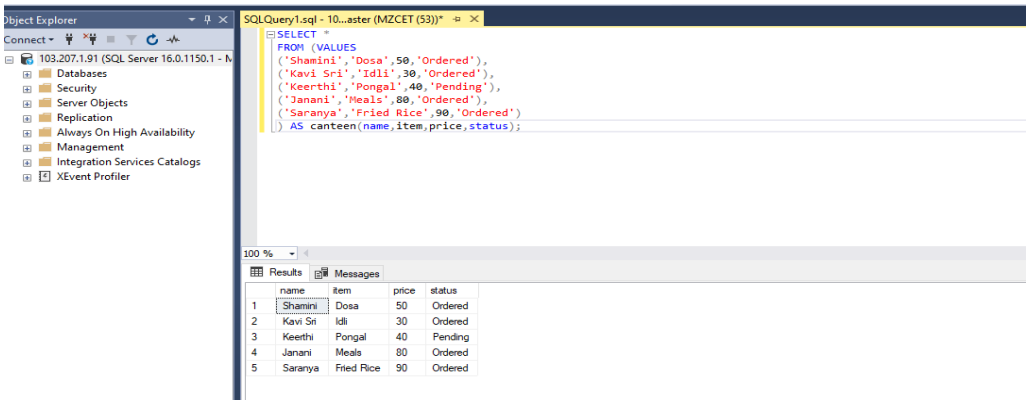
MAIN APPLICATION

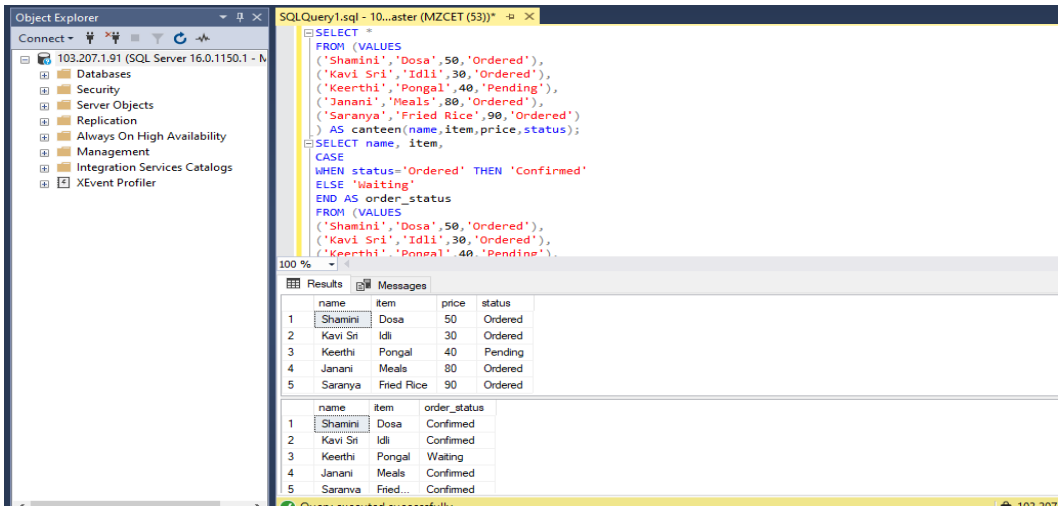
```java id="p4h3na"
public class MainApp {
    public static void main(String[] args) {
        Order o = new Order();
        o.placeOrder("Shamini", "Dosa", 50);
```

NAME: SHAMINI C
SPR NO: 9490

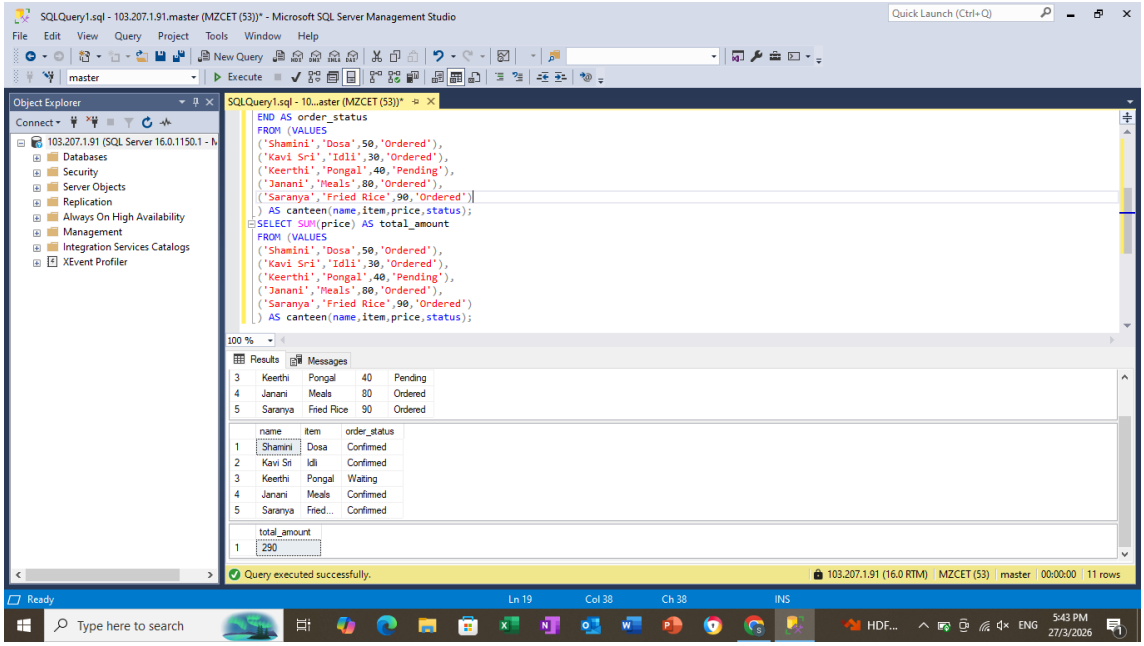
```
    }

}
```

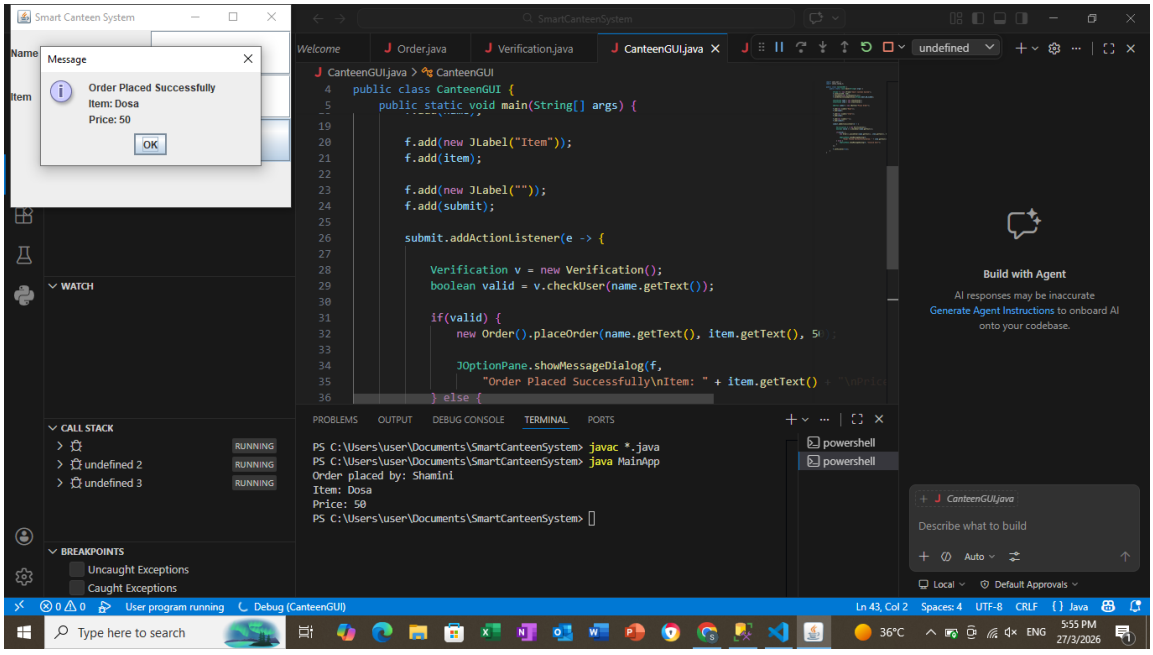
OUTPUT:





NAME: SHAMINI C
SPR NO: 9490





RESULT:

Thus, the Smart Canteen Management System was successfully implemented as per the detailed design. The system manages food orders, verifies users, and calculates total cost efficiently.

NAME: SHAMINI C
SPR NO: 9490

EXP NO: 8 - TEST THE SOFTWARE SYSTEM FOR ALL THE SCENARIOS IDENTIFIED AS PER THE USE CASE DIAGRAM

PROJECT TITLE: Smart Canteen Management System

1. TESTING APPROACH

- The system is tested using **black-box testing techniques**.
- All use case scenarios like viewing menu and placing orders are verified.
- Both valid and invalid inputs are tested.
- Testing ensures correctness, reliability, and performance of the system.

2. TEST CASES FOR USE CASES

Use Case 1: View Menu

Test Case 1.1: Successfully Display Menu

- Input: Select option 1
- Expected Output: List of food items displayed
- Result: Pass

Use Case 2: Place Order

Test Case 2.1: Successfully Place Order

- Input: Valid item name and quantity
- Expected Output: Order placed and total displayed
- Result: Pass

Test Case 2.2: Invalid Item Name

- Input: Wrong item name

- Expected Output: “Item not found” message
- Result: Pass

Test Case 2.3: Invalid Quantity

- Input: Quantity = 0 or negative
- Expected Output: Error message
- Result: Pass

Use Case 3: Exit System

Test Case 3.1: Exit Application

- Input: Select option 3
- Expected Output: Program terminates
- Result: Pass

3. UNIT TESTING

- Individual modules like menu display and order processing are tested separately.
- Each function is verified for correct input and output.
- Errors are identified and corrected at module level.

4. INTEGRATION TESTING

- Interaction between modules (Java program and database) is tested.
- Verified that menu data is retrieved correctly from database.
- Ensured orders are inserted properly into database.

NAME: SHAMINI C
SPR NO: 9490

5. SYSTEM TESTING

- Entire system is tested as a whole.
- All functionalities work together correctly.
- System performance and user interaction are validated.

6. TEST SUMMARY

Test Type	Status
Unit Testing	Passed
Integration Testing	Passed
System Testing	Passed

RESULT

Thus, the Smart Canteen Management System was successfully tested for all the identified scenarios. The system correctly performs menu display, order placement, billing, and database management as per the use case diagram.

NAME: SHAMINI C
SPR NO: 9490

EXP NO: 09 - IMPROVE THE REUSABILITY AND MAINTAINABILITY OF THE SOFTWARE SYSTEM BY APPLYING APPROPRIATE DESIGN PATTERNS

PROJECT TITLE: Smart Canteen Management System

DESIGN PATTERNS USED

- Singleton Pattern – Database Connection
- Factory Pattern – User Creation (Admin, Customer, Staff)
- Observer Pattern – Order Notification System

1. SINGLETON PATTERN (DBCONNECTION)

Ensures only one database connection is created throughout the application.

```
import java.sql.*;


public class DBConnection {

    private static Connection con;

    private DBConnection() {}

    public static Connection getConnection() {

        try {

            if (con == null) {

                Class.forName("com.microsoft.sqlserver.jdbc.SQLServerDriver");

                con = DriverManager.getConnection(

"jdbc:sqlserver://127.0.0.1:1433;databaseName=canteen_db;user=sa;password=1234;encrypt
=true;trustServerCertificate=true"

                );
```

```
            System.out.println("Connection Created");
        }
    } catch (Exception e) {
        System.out.println(e);
    }
    return con;
  }
}
```

2. FACTORY PATTERN (USER FACTORY)

Used to create different types of users dynamically.

Code:

```
public class UserFactory
  public static User getUser(String type, String name)
    if (type.equalsIgnoreCase("customer"))
        return new Customer(name);
    else if (type.equalsIgnoreCase("staff"))
        return new Staff(name);
    else if (type.equalsIgnoreCase("admin"))
        return new Admin(name);
    return null;
  }
}
```

NAME: SHAMINI C
SPR NO: 9490

3. OBSERVER PATTERN (ORDER NOTIFICATION SYSTEM)

Used to notify users when order is placed or completed

Observer Interface

```
public interface Observer {
    void update(String message);
}
```

Customer Observer

```
public class CustomerObserver implements Observer {
    String name;
    public CustomerObserver(String name) {
        this.name = name;
    }
    public void update(String message) {
        System.out.println("Notification to " + name + ": " + message);
    }
}
```

Order System (Subject)

```
import java.util.*;
public class OrderSystem {
    List<Observer> observers = new ArrayList<>();
    public void addObserver(Observer o) {
        observers.add(o);
    }
```

```
    public void placeOrder() {
        notifyObservers("Order Placed Successfully");
    }
    public void completeOrder() {
        notifyObservers("Order Ready for Pickup");
    }
    public void notifyObservers(String msg) {
        for (Observer o : observers) {
            o.update(msg);
        }
    }
}
```

4. USER CLASSES

Abstract Class

```
public abstract class User {
    String name;
    public abstract void role();
}
```

Customer

```
public class Customer extends User {
    public Customer(String name) {
        this.name = name;
```

```
    }
    public void role() {
        System.out.println(name + " is Customer");
    }
}
```

Staff

```
public class Staff extends User {
    public Staff(String name) {
        this.name = name;
    }
    public void role() {
        System.out.println(name + " is Staff");
    }
}
```

Admin

```
public class Admin extends User {
    public Admin(String name) {
        this.name = name;
    }
    public void role() {
        System.out.println(name + " is Admin");
    }
}
```

NAME: SHAMINI C
SPR NO: 9490

5. MAIN PATTERN DEMO

```
import java.sql.*;

public class MainPatternDemo {

    public static void main(String[] args) {

        Connection c1 = DBConnection.getConnection();

        Connection c2 = DBConnection.getConnection();

        if (c1 == c2) {

            System.out.println("Singleton working");

        }

        User u1 = UserFactory.getUser("customer", "Shamini");

        User u2 = UserFactory.getUser("admin", "Manager");

        u1.role();

        u2.role();

        OrderSystem o = new OrderSystem();

        CustomerObserver cst1 = new CustomerObserver("Shamini");

        CustomerObserver cst2 = new CustomerObserver("Kavi");

        o.addObserver(cst1);

        o.addObserver(cst2);

        o.placeOrder();

    }

}
```

NAME: SHAMINI C
SPR NO: 9490

OUTPUT:

Connection Created

Singleton working

Shamini is Customer

Manager is Admin

Notification to Shamini: Order Placed Successfully

Notification to Kavi: Order Placed Successfully

RESULT:

Thus, appropriate design patterns such as **Singleton, Factory, and Observer** were successfully applied to improve the **reusability and maintainability** of the Smart Canteen Management System.

NAME: SHAMINI C
SPR NO: 9490

EX. NO: 10 - IMPLEMENT THE MODIFIED SYSTEM AND TEST IT FOR VARIOUS SCENARIOS

PROJECT TITLE: Smart Canteen Management System

STEP 1: Integrate Design Patterns

Design Pattern Used

Singleton Design Pattern

Purpose

- Ensures only one database connection is used throughout the system
- Improves performance and resource utilization

Implemented Features

- Centralized database connection module
- Menu retrieval using shared connection
- Order placement using same connection
- Reduced redundancy in database operations
- Improved system efficiency and maintainability

STEP 2: Regression Testing

Regression testing ensures that newly added features do not affect existing functionalities of the system.

NAME: SHAMINI C
SPR NO: 9490

TEST PLAN

Test Case ID	Test Case	Input	Expected Output	Result
TC-01	View Menu	Option 1	Menu displayed	Pass
TC-02	Place Order	Valid item	Order success	Pass
TC-03	Invalid Item	Wrong name	Error message	Pass
TC-04	Quantity Check	0 or negative	Not allowed	Pass
TC-05	Exit	Option 3	Program ends	Pass

Testing Tools Used

- Java Console (for execution)
- SQL Server Management Studio (for database verification)
- Manual Testing Method

STEP 3: Generate Final Report

Summary of Modifications:

- Design Patterns Applied: Improved reusability and maintainability.
- Testing Performed: Unit Testing, Integration Testing, Regression Testing.
- Final Outcome: The smart Canteen Management System is stable, efficient, and easy to maintain

RESULT

Thus, the modified smart canteen management System was successfully implemented and tested across various scenarios.